

J-ART: A Japanese Adversarial Red-Team Framework for Application-Layer LLM Security and Cost-Efficiency

Technical Report (v0.2, preprint)

Takayuki Hirose*

2026-06-14

Abstract

Most open red-teaming and jailbreak benchmarks for large language models (LLMs) are English-centric and evaluate *bare models*. We present **J-ART** (Japanese Adversarial Red-Team framework), an open, reproducible harness that (i) evaluates the **application-structure layer** — model $\times$ system-prompt strength $\times$ guardrail $\times$ retrieval (RAG) — rather than bare models; (ii) applies **Japanese-specific obfuscation transforms** (gyaru-moji character substitution, vertical/newline splitting, keigo “polite-coercion” framing, double-talk false premises, Base64, and leet/zero-width smuggling) that defeat naïve keyword filters; (iii) grounds attacks in **MITRE ATLAS** (10 inference-time techniques); and (iv) reports a **cost-efficiency** metric alongside a defense rate with **Wilson 95% confidence intervals**. Attacks use *harmless proxy markers* (a fictitious canary and benign output markers); the framework contains **no weaponizable content**, and the malicious “core” of each attack is masked in all published artifacts. Across 21 application configurations and 11 attacks $\times$ 7 transforms (live, $K = 5$ trials per cell), we find that **application-layer defenses dominate raw model capability** — naked models range 29.6–86.5% defense, but a balanced ablation on **three** base models with **cluster-robust** intervals shows a **hardened prompt and model-based guardrails significantly raise defense on every model**, while lightweight keyword/regex filters help most where the base model is weak (the effect scales inversely with the naked baseline) — and that a simple **normalizing filter** defeats the obfuscation transforms a naïve keyword filter misses. We also document **large within-campaign variance**: in a controlled $K = 5$ repeat of a frozen configuration set, **9.6% of cells changed outcome across identical trials**, which makes single-sample leaderboards unreliable and motivates our statistical treatment.

Status. This is a technical report / preprint describing the framework and its live findings. It is *not* an official safety evaluation of any vendor’s model; numbers are run-time snapshots (2026-06-14) subject to Section 7, and all defense rates are *lexical-leak* upper bounds. The §4 numbers reproduce from the committed artifacts named in Section 9. References are real, canonical works; final venue/year/DOI formatting will be completed for camera-ready.

1 Introduction

Public LLM red-teaming tooling (e.g., garak [2]) and jailbreak benchmarks (e.g., HarmBench [3], JailbreakBench [4]) are predominantly English. Multilingual safety work has shown that

*Independent Researcher. ORCID: 0009-0005-6735-1862. Code: <https://github.com/takker-hero-se/J-ART>. Live leaderboard: <https://takker-hero-se.github.io/J-ART/>. Software DOI (concept): 10.5281/zenodo.20676879. Report text licensed CC-BY-4.0; code MIT.

translating prompts into low-resource languages can bypass safety training, but **language-specific obfuscation** — exploiting a language’s own scripts, typography, and register — remains comparatively underexplored. Japanese in particular offers a rich obfuscation surface (multiple scripts, full-width/zero-width characters, vertical writing, and conventionalized polite registers).

Two further gaps motivate J-ART. First, **bare-model versus deployed-system evaluation**: practitioners deploy a model *plus* a system prompt, *plus* guardrails, often *plus* retrieval; the security-relevant unit is the **application configuration**, not the model. Second, **security and cost jointly**: defenses cost tokens or latency, so we report a cost-efficiency score so that “cheapest sufficiently-safe configuration” is directly readable.

Our contributions are: (a) a Japanese obfuscation transform suite; (b) an application-layer, MITRE-ATLAS-grounded harness with a safe proxy-marker methodology; (c) a guardrail comparison including a normalizing filter and a model-based classifier; (d) a statistically-reported leaderboard (Wilson CIs, optional repeated trials) with a cost-efficiency metric; and (e) an empirical observation of large router-induced run-to-run variance.

2 Related Work

J-ART sits at the intersection of six lines of work, summarized below and positioned in Table 1.

Adversarial red-teaming tooling. garak [2] scans a *model* with a library of vulnerability probes. J-ART instead treats the deployed *application configuration* (model $\times$ system prompt $\times$ guardrail $\times$ RAG) as the unit under test.

Jailbreak and harmful-behavior benchmarks. AdvBench/GCG [11], Jailbroken [12], HarmBench [3], JailbreakBench [4], and in-the-wild prompts [13] standardize *which* harmful behaviors to elicit and *how* to score them, predominantly in English at the model layer. J-ART reuses the harness idea but replaces harmful elicitation with a safe proxy-marker objective (§3) so the suite can be released publicly.

Automated attack generation. PAIR [6] and TAP [7] optimize jailbreak prompts against a target. J-ART uses a *fixed, auditable* transform suite, trading attack strength for reproducibility and safe disclosure.

Multilingual and low-resource safety. Deng et al. [14] and Yong et al. [5] show alignment degrades under *translation* into low-resource languages, and Wang et al. [15] study cross-lingual safety generalization. These study *translation across* languages; J-ART studies *intra-language orthographic obfuscation* specific to Japanese (gyaru-moji, vertical writing, keigo framing, kana/kanji encoding), which persists even though Japanese is high-resource.

Encoding/typographic obfuscation. Encoding-based jailbreaks are established prior art: CipherChat [28] (ciphered conversation) and ArtPrompt [29] (ASCII-art typography). Our `base64_wrap` and `leet_smuggle` transforms are **language-agnostic replications** of this line, used only as a baseline; the novel surface is the Japanese-script transforms (§3).

Prompt injection and application-layer evaluation. Perez & Ribeiro [16] introduced goal-hijacking; Greshake et al. [8] formalized *indirect* injection; Liu et al. [17] benchmark LLM-integrated apps; AgentDojo [30] is the closest application-layer evaluation (tool-using agents). J-ART instead holds the *application configuration* (prompt $\times$ guardrail $\times$ RAG) as the explicit unit of evaluation against a Japanese-obfuscation surface.

Table 1: Positioning of J-ART relative to representative prior work.

Work	Language	Unit of eval.	Cost axis	Guard/RAG	Public
garak [2]	English	Model	No	No	Yes
HarmBench / JBB [3, 4]	English	Model	No	No	Yes
PAIR / TAP [6, 7]	English	Model	No	No	Partial
Multiling. [5, 14, 15]	Low-resource	Model	No	No	Yes
Llama Guard / NeMo [9, 18]	Eng.-centric	Classifier	No	n/a	Yes
J-ART	Japanese	App config	Yes	Yes	Yes

Guardrails and input classifiers. Llama Guard [9], NeMo Guardrails [18], and the OpenAI Moderation API [19] provide model- or rule-based filtering. J-ART treats the guardrail as a *swappable configuration variable* and measures the marginal defense each adds, including a deterministic normalizing filter that defeats obfuscations a naive keyword filter misses (§4).

LLM-as-judge validation. Zheng et al. [20] and the survey of Gu et al. [21] document the biases of LLM judges. J-ART avoids an LLM judge for its *primary* outcome, using a deterministic canary/marker check (§3) so judge variance does not enter the headline metric.

Japanese NLP resources and taxonomies. General Japanese benchmarks such as JGLUE [22] cover NLU, but to our knowledge no open, reproducible *adversarial-safety* benchmark targets Japanese-specific obfuscation — the gap J-ART addresses. We map attacks to MITRE ATLAS [1] and cross-reference the OWASP Top 10 for LLM Applications [10].

J-ART is thus distinguished by combining a *Japanese intra-language obfuscation* surface, an *application-configuration* unit of evaluation, a *cost-efficiency* axis, and a *safe proxy-marker* methodology in a single reproducible harness.

3 Methodology

Application-structure layer. Each evaluated **target** is a configuration: provider/model $\times$ prompt_strength $\in \{\text{nailed (low), hardened (high)}\} \times$ guardrail $\in \{\text{none, keyword, regex, llm, llamaguard}\} \times$ rag $\in \{\text{true, false}\}$. A simplified RAG context (benign corporate documents, optionally poisoned for indirect-injection attacks) and a system prompt containing a secret *canary* are assembled per trial.

Japanese obfuscation transforms. Each base attack is rendered through one of seven transforms: **baseline** (none), **polite_business** (keigo authority framing), **vertical_newline** (one-character-per-line vertical writing), **gyaru** (gyaru-moji glyph substitution + zero-width spaces), **double_tongue** (false sandbox/authorization premise), **base64_wrap** (Base64-encoded instruction), and **leet_smuggle** (leet symbol substitution + zero-width spaces). Transforms preserve human legibility while breaking substring-matching filters.

We distinguish two classes (Table 2). *Japanese-specific* transforms exploit the Japanese writing system or register and have no direct English equivalent: **gyaru** (kana glyph substitution), **vertical_newline** (vertical writing), and **polite_business/double_tongue** (keigo authority and in-group framing). *Language-agnostic* transforms — **base64_wrap** and **leet_smuggle**, plus the **baseline** control — are **not novel**: they replicate the encoding/typographic jailbreak line

Table 2: Classification of obfuscation transforms.

Class	Transforms	Exploits
Japanese-specific	<code>gyaru</code> , <code>vertical_newline</code> , <code>polite_business</code> , <code>double_tongue</code>	Kana glyph variants, vertical writing, keigo authority/in-group framing
Language-agnostic	<code>base64_wrap</code> , <code>leet_smuggle</code> (+ <code>baseline</code>)	Encoding / symbol substitution applicable to any language

(CipherChat [28], ArtPrompt [29]) and serve only as a baseline so the marginal effect of the Japanese-specific surface can be isolated.

Attack suite and MITRE ATLAS mapping. Eleven attacks cover ten inference-time ATLAS techniques: Direct/Indirect Prompt Injection (AML.T0051.000/.001), LLM Jailbreak (AML.T0054), LLM Data Leakage (AML.T0057), LLM Prompt Self-Replication (AML.T0061), Trusted-Output/Citations manipulation (AML.T0067.000), LLM Prompt Obfuscation (AML.T0068), System-Prompt discovery (AML.T0069.002), RAG Poisoning (AML.T0070), and False RAG Entry Injection (AML.T0071). Vectors are `user` (direct) or `rag` (indirect).

Harmless proxy markers and safety-by-construction. To avoid producing weaponizable content, attacks target **harmless proxies**: a fictitious canary passphrase embedded in the system prompt (data-leakage attacks) and benign string markers [JART-PWNED] / [JART-HIJACKED] (injection / control-hijack attacks). The malicious instruction **core** of each attack is defined in code, kept out of configuration files, and **masked** (replaced by a redaction placeholder) in `results.json` and the public site; only the obfuscation *framework* is shown, so developers can study the structure without a usable recipe. A defense-in-depth client-side mask is also applied at render time.

Deterministic judge. A breach is declared by membership — the canary or marker in the response, matched exactly *and* after light normalization. This judge detects a *lexical* leak and cannot detect a semantic/paraphrased one, so we call the metric a **lexical-leak defense rate**: every defense rate in this paper is an **upper bound** on true safety, not a certificate. A human-validated judge is future work.

Guardrails. Five settings are compared: **none**; **keyword** (banned-substring match — fast, weak to obfuscation); **regex** (a *normalizing* filter that strips zero-width characters, reverses leet substitutions, folds whitespace, and Base64-decodes long tokens *before* keyword matching — deterministic, zero extra API cost); **llm** (the target model self-moderates input and screens output); and **llamaguard** (a dedicated safety-classification model — Llama Guard 4 via a model router — classifies the input as attack/benign, with a graceful fallback on failure).

Statistical treatment. Each defense rate carries a **Wilson 95% confidence interval** over its trial count; each cell (target, attack, transform) is repeated N times (JART_TRIALS). The trial-level Wilson interval treats the K repeats of a cell as independent, which they are **not**. For any *difference* between configurations (the ablation, §4) we therefore report a **cluster-robust bootstrap 95% CI** that resamples whole *cells* (`bootstrap_diff_ci`, 3,000 resamples, fixed seed); these are wider than the trial-level Newcombe intervals and are the intervals on which every significance claim rests.

Cost-efficiency metric. We report $\text{cospa} = \text{defense_rate}(\%) \div \text{cost_per_million_tokens (USD)}$ (per-million cost clamped at a \$0.01 floor). *cospa* is a deliberately simple *screening ratio*, not a utility model: its units are not independently meaningful and the ranking is sensitive to the clamp and price snapshot. We treat it only as a sort key and present the underlying **defense-vs-cost trade-off** directly; the clamp affects only sub-\$0.01/Mtok configurations, of which our fleet has none.

Execution. Trials are independent and run with bounded concurrency (a thread pool, default 8 workers) under per-request timeouts and exponential-backoff retries; determinism of the simulation does not depend on execution order. Targets whose API key is absent fall back to a deterministic simulation so a partial key set never aborts the run.

4 Results

We report a fully-labeled **live campaign** (21 configurations $\times$ 11 attacks $\times$ 7 transforms $\times$ $K = 5$ trials = 8,085 trials), executed 2026-06-14. Every cell is labeled **mode=LIVE** with a price snapshot; a trial is excluded iff **api_error** (at the trial level); after parity re-runs this removes 169 of 8,085, leaving 7,916 **clean trials**; billed cost over clean cells (campaign plus parity re-runs, including the expensive Opus flagship) is $\approx$ \$6.0. **Opus naked is now fully measured** ($n=385$, after an Anthropic top-up); **only Gemini-pro remains partial** ($n=227$; per-minute rate limits) and is treated as *provisional*. All defense rates are **lexical-leak** upper bounds (§3); every difference carries a **cluster-robust** CI (§3). Each §4 number is reproducible from committed artifacts (§9).

Overall. All configurations include the simplified RAG context (§3); **“naked” means no system-prompt hardening and no guardrail, not “no RAG”** (indirect-injection attacks are in scope even for the naked baselines). The aggregate breach rate over clean trials was **14.5%** (1,151/7,916), **concentrated in weak configurations**. Every hardened or guardrailed configuration, on *any* model, defended in the **91–100%** band, whereas bare models ranged from **29.6% to 86.5%** (Table 3).

Application-layer defense vs. model capability. We isolate the configuration effect from model capability with a balanced ablation on **three** base models spanning the range (weak Llama-4-Scout; cheap GPT-4o-mini; flagship GPT-4.1). Each holds the model fixed and crosses {naked, hardened} prompt $\times$ guardrail; we report each config’s difference from that model’s naked baseline with a **cluster-robust bootstrap 95% CI** (3,000 cell-resamples; resampling cells, not trials, so the within-cell correlation of the ablation’s $K=3$ repeats does not inflate significance; reproduced by `experiments/analyze_ablation.py`; Table 4). Two robust patterns hold across all three models: (1) a **hardened prompt** and the **model-based guardrails** (Llama Guard, LLM self-moderation) significantly raise defense on *every* model; (2) the effect **scales inversely with the naked baseline** — on weak Scout (naked 38%) every guardrail adds +26 to +60 points, whereas on GPT-4o-mini (naked 82%, near the ceiling) the lightweight keyword/regex filters are *not* individually significant under clustering. Configuration matters most exactly where the base model is weakest. The thesis — *configuration matters more than model choice* — rests on a clean, fully-measured comparison: a hardened cheap model (GPT-4o-mini hardened $\approx$ 100%) **out-defends a naked flagship** (GPT-4.1 naked 64.1%, $n=231$) by $\sim$ 36 points, and likewise out-defends a naked Opus (86.5%, $n=385$). Even at full parity the strongest naked model trails a hardened cheap one.

Within-campaign trial variance (a primary result). Within a single campaign, every cell of a frozen configuration set is sampled $K = 5$ times. Over the **1,577 cells with all**

Table 3: Naked-model defense (no prompt, no guardrail; $K = 5$).

Naked model	Defense	95% CI	n
DeepSeek-V3	29.6%	[25.3, 34.4]	385
Llama 4 Scout	37.9%	[33.2, 42.9]	385
Gemini 2.5 Flash	49.6%	[44.6, 54.6]	385
gpt-oss-120b	82.6%	[78.5, 86.1]	385
GPT-4o-mini	83.9%	[79.9, 87.2]	385
Claude Opus 4.x	86.5%	[82.7, 89.5]	385

Table 4: Configuration effect (Δ vs. that model’s naked baseline; cluster-robust 95% CI). $\times$ = not significant.

Added defense (low-prompt)	Scout (38.1%)	4o-mini (82.3%)	GPT-4.1 (64.1%)
+ keyword	+26.0 [+13,+39]	+9.1 [−1,+20] $\times$	+12.5 [−1.5,+26] $\times$
+ regex	+45.0 [+33,+56]	+10.0 [+0,+20] $\times$	+23.8 [+11,+36]
+ Llama Guard	+47.2 [+36,+58]	+15.6 [+7,+25]	+25.1 [+13,+38]
+ LLM guardrail	+59.7 [+51,+68]	+17.7 [+10,+26]	+35.9 [+26,+47]
hardened prompt	+47.2 [+36,+58]	+17.7 [+10,+26]	+33.3 [+23,+45]

five trials clean, 9.6% (152/1,577) changed outcome across the five identical repeats, concentrated in naked weak models. For “Llama 4 Scout / naked” — the original cautionary example — of its 77 cells, **32 (42%) were unstable**: 14 never breached (0/5), 31 always (5/5), and 32 in between. A *single-sample* leaderboard would assign this configuration anywhere from 0% to 100% by chance — controlled evidence (free of the configuration-set confound) that **a single sample per cell is not reproducible**. We scope this to *within-campaign* (provider non-determinism); across-day drift is scaffolded (`across_runs`) but left to future work.

Guardrail comparison and cost. Across the three ablation models (Table 4) the ordering is consistent: the LLM guardrail and a hardened prompt give the largest gains, Llama Guard next, and lightweight keyword/regex help most on weak base models. Guardrails that block an input **skip the main model call**, lowering net cost while raising defense; the **normalizing regex filter** (verified in unit tests) adds +45 points on weak Scout at zero API cost. On the cost-efficiency metric, leaders are inexpensive OSS models behind a lightweight guardrail (gpt-oss-20b + keyword, $\text{cospa} \approx 1131$), while a flagship naked configuration is worst (Opus naked, $\text{cospa} \approx 6.8$) — three orders of magnitude apart.

Attack and transform effectiveness. The most effective attacks were Trusted-Output/Citations manipulation, False RAG Entry Injection, and System-Prompt discovery. Among transforms, the *un-obfuscated* baseline was most effective (**22.3%**), then *vertical_newline* (20.2%) and *polite_business* (17.6%), while **base64_wrap** was **least effective (5.9%)**: heavy encoding makes models refuse or fail to parse the instruction more often than it smuggles it past them. The most dangerous Japanese transforms are the *legible* ones (vertical writing, keigo framing), not the cryptographic ones.

5 The Leaderboard Artifact

The harness emits `results.json` and a self-contained static site (sortable leaderboard, per-configuration attack logs with masked prompts, JP/EN UI with environment-based auto-selection, per-model defense-rate CIs, and per-cell breach rates), continuously deployed.

Table 5: Governance crosswalk (informational).

J-ART (ATLAS)	capability	OWASP LLM Top 10	NIST RMF	AI	EU AI Act	Japan
Prompt injection / jailbreak (T0051, T0054)		LLM01	MEASURE 2.7		Art. 15, 55	AISI; AI Biz
Canary / data leakage (T0057)		LLM02	MEASURE 2.7		Art. 15, 10	AI Biz
System-prompt discovery (T0069.002)		LLM07	MEASURE 2.7		Art. 15	AISI
RAG poisoning / false entry (T0070, T0071)		LLM08; LLM01	MEASURE 2.7		Art. 15, 10	AISI
Trusted-output manipulation (T0067.000)		LLM09; LLM05	MEASURE 2.6		Art. 50	AI Biz
Obfuscation / self-replication (T0068, T0061)		LLM01	MEASURE 2.7		Art. 55	AISI

6 Ethics and Responsible Disclosure

Safety by construction. J-ART is safe by construction. (1) Attacks target a **fictitious canary** and **benign markers**, never real harmful capabilities; no weaponizable content exists in the code, data, or published site. (2) The malicious instruction **core** is masked in all artifacts; only the obfuscation framework is shown. (3) Models are referenced for research comparison only; results are run-time snapshots, not certifications. The intended use is to help developers harden their *own* deployments.

A reusable disclosure pattern. We state the design above as a transferable pattern for publishing adversarial safety benchmarks without releasing operational attacks: (i) a *harmless proxy objective* (a fictitious canary and benign markers) so a breach is measurable but inert; (ii) a *core/framework split with masking* — publish the obfuscation framework that carries the research signal, mask the operational core in every artifact, and keep harmless cores only in code; (iii) a *deterministic offline mode* so results reproduce from the repository without distributing a working exploit corpus; and (iv) *audited masking* — a unit-test invariant checks that no core (raw or obfuscated) reaches a published artifact. This keeps the methodology and aggregate findings fully open while the attack payloads stay withheld — the posture we recommend for language-specific red-team datasets generally.

Governance crosswalk. Table 5 cross-references each evaluated capability to MITRE ATLAS [1], the OWASP Top 10 for LLM Applications [10], the NIST AI RMF [23] and its Generative-AI Profile [24], the EU AI Act [25], and Japan’s AI governance guidance [26, 27]. The mapping is **informational, not a compliance claim**; applicability depends on each deployment’s risk classification. Operationally J-ART is a **MEASURE**-stage activity in the NIST AI RMF whose outputs inform an organization’s **MANAGE** decisions.

Worked example. Under NIST MEASURE 2.7, an organization could record a J-ART line item as “prompt-injection resilience (T0051/T0054), lexical-leak defense rate 93.5% (Wilson 95% CI [90.6, 95.6] for this single rate; the cluster-robust interval is for *differences*, §3), $n=385$, $K=5$, GPT-4.1-mini + hardened prompt + keyword guardrail, 2026-06-14” with a pre-registered threshold (e.g. “CI lower bound $\geq 90\%$ ”); the same row is the evidence an AISI red-team report or an EU AI Act Art. 15 dossier would cite.

Dual-use. The published transform suite is also a reusable attack wrapper. The attacker uplift is low (the Japanese-script transforms are folklore; the encodings replicate public work; no harmful payloads are included), while the defender benefit is concrete: the same repository ships the **normalizing regex guardrail** that neutralizes every transform (unit-tested). We judge open release net-positive for defenders, consistent with §6.2.

7 Limitations

- **Proxy markers, not harm:** we measure instruction-violation / leakage proxies rather than real-world harm.
- **Deterministic judge:** the judge matches the canary/marker exactly *and* after light normalization (case-folding, whitespace/zero-width removal), so a reformatted echo is counted as a breach. It cannot detect *semantic*/paraphrased leakage, so reported defense rates are an **upper bound**; a human-validated LLM-judge is future work.
- **Single-turn:** multi-turn / crescendo attacks are not yet modeled.
- **Sample size:** a handcrafted suite (11 attacks $\times$ 7 transforms); default $N=1$ per cell.
- **Provider non-determinism / router variance:** results are snapshots and depend on routing and model versions at run time.
- **Non-random missingness (now mostly resolved):** we flagged that an earlier campaign’s partial configs had **non-random** missingness — rate limits correlate with attack difficulty, under-representing Opus’s hardest attacks. Parity re-runs **confirmed this directly**: Opus naked at full $n=385$ defends **86.5%**, vs. the survivorship-biased **89.6%** on $n=125$ — a ~ 3 -point over-estimate, as predicted. Qwen3-max and Opus are now at parity; **only Gemini-pro remains partial** ($n=227$; per-minute rate limits a credit top-up cannot raise) and is treated as provisional, with no headline resting on it.
- **Cost attribution:** guardrail tokens are now priced at the *guardrail model’s* own rate (the classifier JART_GUARD_MODEL for llamaguard; the target model itself for the llm guardrail, which genuinely calls it) and reported separately. Remaining approximation: simulated token *counts* are estimated.
- **Simulation mode** is a deterministic stand-in for missing keys and must not be read as empirical model behavior.

8 Future Work

LLM-judge with human-validated agreement; multi-turn and adaptive (PAIR/TAP-style) attacks; larger and partially human-curated attack sets; additional ATLAS techniques (cost-exhaustion, model extraction); per-provider concurrency and incremental caching with longitudinal trend tracking; and dropping simulation mode for a purely empirical, repeated-trial statistical report.

9 Reproducibility

Code, configuration, and the deployment workflow are public. The harness is deterministic in simulation mode; live results depend on provider routing and model versions at run time and should be reported with the run date and model identifiers. See the repository for exact model identifiers and pricing used. Each cell carries provenance labels — `mode` (LIVE/MOCK), `api_error`, the resolved model, and a `price_per_million` snapshot — and simulated cells are excluded from the live aggregates. **Every \$4 number is reproducible from committed artifacts:**

`experiments/var_run.json` (§4.1 overall, naked-spread, variance, transforms; cospa); and the three ablation files `ablation_results.json` (GPT-4o-mini), `abl_scout_results.json` (Llama-4-Scout), `abl_gpt41_results.json` (GPT-4.1) for Table 4. Aggregates and cluster-robust CIs are recomputed by `experiments/analyze_ablation.py` and `analyze_variance.py`. The public leaderboard is regenerated by a manual or weekly LIVE CI run; push-triggered runs use simulation mode and are labeled as such, so the site never mixes MOCK and LIVE.

References

- [1] MITRE ATLAS — Adversarial Threat Landscape for AI Systems. <https://atlas.mitre.org> (accessed 2026-06-14).
- [2] garak — Generative AI Red-teaming & Assessment Kit.
- [3] HarmBench: A Standardized Evaluation Framework for Automated Red Teaming.
- [4] JailbreakBench: An Open Robustness Benchmark for Jailbreaking LLMs.
- [5] Yong, Menghini, Bach. Low-Resource Languages Jailbreak GPT-4. 2023.
- [6] Chao et al. Jailbreaking Black-Box LLMs in Twenty Queries (PAIR).
- [7] Mehrotra et al. Tree of Attacks: Jailbreaking Black-Box LLMs Automatically (TAP).
- [8] Greshake et al. Not What You’ve Signed Up For: Indirect Prompt Injection. 2023.
- [9] Inan et al. Llama Guard: LLM-Based Input-Output Safeguard for Human-AI Conversations. Meta, 2023.
- [10] OWASP Top 10 for Large Language Model Applications (2025). <https://genai.owasp.org/llm-top-10/> (accessed 2026-06-14).
- [11] Zou et al. Universal and Transferable Adversarial Attacks on Aligned Language Models (AdvBench/GCG). 2023.
- [12] Wei, Haghtalab, Steinhardt. Jailbroken: How Does LLM Safety Training Fail? 2023.
- [13] Shen et al. “Do Anything Now”: Characterizing and Evaluating In-the-Wild Jailbreak Prompts on Large Language Models. 2024.
- [14] Deng et al. Multilingual Jailbreak Challenges in Large Language Models. 2024.
- [15] Wang et al. All Languages Matter: On the Multilingual Safety of Large Language Models. 2024.
- [16] Perez & Ribeiro. Ignore Previous Prompt: Attack Techniques for Language Models. 2022.
- [17] Liu et al. Prompt Injection Attack against LLM-Integrated Applications. 2023. arXiv:2306.05499.
- [18] Rebedea et al. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. 2023.
- [19] Markov et al. A Holistic Approach to Undesired Content Detection in the Real World (OpenAI Moderation). 2023.
- [20] Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. 2023.

- [21] Gu et al. A Survey on LLM-as-a-Judge. 2024.
- [22] Kurihara et al. JGLUE: Japanese General Language Understanding Evaluation. 2022.
- [23] NIST. AI Risk Management Framework (AI RMF 1.0), NIST AI 100-1. 2023.
- [24] NIST. Artificial Intelligence Risk Management Framework: Generative AI Profile, NIST AI 600-1. 2024.
- [25] European Union. Regulation (EU) 2024/1689 (Artificial Intelligence Act). Official Journal of the EU, 2024.
- [26] METI / MIC (Japan). AI Guidelines for Business, Ver. 1.0. 2024.
- [27] Japan AI Safety Institute (AISI). Guide to Red Teaming Methodology on AI Safety. 2024.
- [28] Yuan et al. GPT-4 Is Too Smart To Be Safe: Stealthy Chat with LLMs via Cipher (CipherChat). 2023.
- [29] Jiang et al. ArtPrompt: ASCII Art-based Jailbreak Attacks against Aligned LLMs. 2024.
- [30] Debenedetti et al. AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents. 2024.